CASE STUDY 6: PASSWORD POLICY VALIDATION

1. Problem Statement

Password Policy Validation is used to check whether a password satisfies the required security rules. The program accepts a password from the user and checks its length, uppercase letters, lowercase letters, digits, and special characters. It displays whether the password is valid or invalid according to the defined rules.

2. Objectives

- To develop a Python program to validate passwords.
- To check the minimum password length.
- To check uppercase and lowercase letters.
- To check whether the password contains a digit and a special character.
- To understand strings, conditional statements, and built-in functions in Python.

3. Assumptions

1. The password must contain at least 8 characters.
2. The password must contain at least one uppercase letter.
3. The password must contain at least one lowercase letter.
4. The password must contain at least one digit.
5. The password must contain at least one special character.
6. A password satisfying all the rules is considered valid.

4. Algorithm

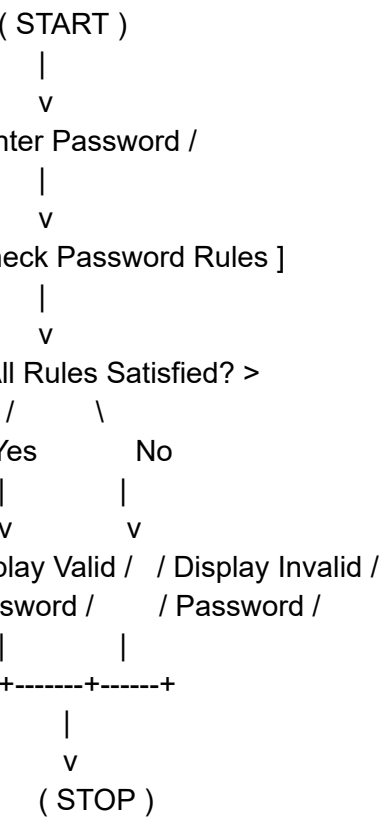
1. Start the program.
2. Accept a password from the user.
3. Check whether the password contains at least 8 characters.
4. Check whether it contains an uppercase letter.
5. Check whether it contains a lowercase letter.
6. Check whether it contains a digit.
7. Check whether it contains a special character.
8. If all conditions are satisfied, display "Valid Password".
9. Otherwise, display "Invalid Password".
10. Stop the program.

5. Flowchart

Flowchart symbols:

- Oval – Start/Stop
- Parallelogram – Input/Output
- Rectangle – Process
- Diamond – Decision
- Arrow – Direction of flow

Simple flowchart:



Draw Start and Stop using oval shapes, input/output using parallelograms, checking rules using a rectangle, and the decision using a diamond.

6. Pseudocode

START

INPUT password

IF length of password >= 8
  AND password contains uppercase letter
  AND password contains lowercase letter
  AND password contains a digit
  AND password contains a special character

  DISPLAY "Valid Password"

```
ELSE
   DISPLAY "Invalid Password"
END IF

STOP
```

7. Python Source Code

```
password = input("Enter password: ")

if (len(password) >= 8
   and any(c.isupper() for c in password)
   and any(c.islower() for c in password)
   and any(c.isdigit() for c in password)
   and any(not c.isalnum() for c in password)):

   print("Valid Password")

else:
   print("Invalid Password")
```

8. Test Cases and Expected Outputs

Test Case 1: Valid Password

Input:

Enter password: Hello@123

Expected Output:

Valid Password

Test Case 2: Password Too Short

Input:

Enter password: Ab@12

Expected Output:

Invalid Password

Test Case 3: Missing Special Character

Input:

Enter password: Hello123

Expected Output:

Invalid Password

Note: Run each test case separately and enter the password exactly as shown.

9. Output Screenshots

```
password = input("Enter password: ")

if (len(password) >= 8
    and any(c.isupper() for c in password)
    and any(c.islower() for c in password)
    and any(c.isdigit() for c in password)
    and any(not c.isalnum() for c in password)):

    print("Valid Password")

else:
    print("Invalid Password")
```

```
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case6.
py
Enter password: Hello@123
Valid Password
>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case6.
py
Enter password: Ab@12
Invalid Password
>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case6.
py
Enter password: Hello123
Invalid Password
>>
```

10. Conclusion

Thus, the Password Policy Validation program was successfully implemented using Python. The program checks password length, uppercase letters, lowercase letters, digits, and special characters. It displays whether a password satisfies the specified rules. This case study demonstrates the use of strings, conditional statements, and built-in functions in Python